JAVA COURSE

{ memoria e stringhe }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.0

INDICE

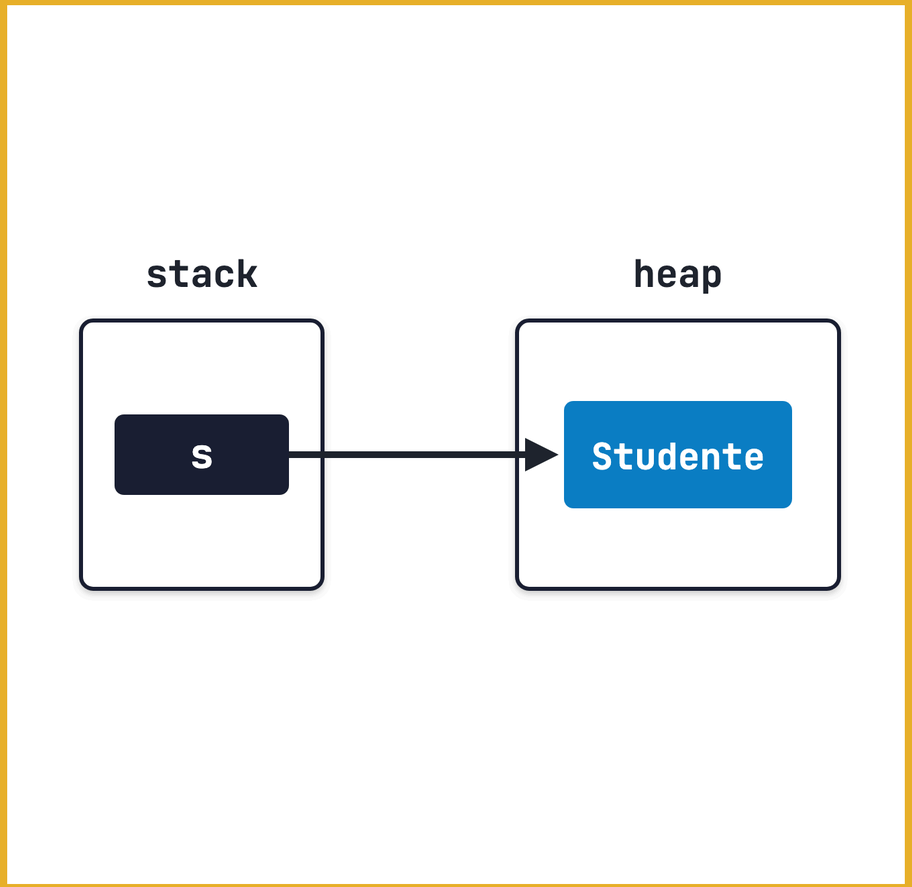
01 La memoria in Java

02 Le stringhe

03 Formattare e convertire

04 Stringhe in pratica

01
LA MEMORIA IN JAVA



PERCHÉ PARLARE DI MEMORIA

Con le classi abbiamo visto alcune regole: `null` causa un errore, le `String` si confrontano con `equals`, un oggetto passato a un metodo cambia anche fuori. Ora vediamo **perché**

- *Dove finisce un oggetto creato con* `new`*?*
- *Perché due variabili a volte si influenzano a vicenda?*
- *Perché* `==` *sulle stringhe non funziona come ci aspettiamo?*

STACK E HEAP

Java divide la memoria in due zone con ruoli distinti

- *Ogni chiamata a metodo crea un **frame** sullo stack, che sparisce quando il metodo termina*
- *Gli oggetti vivono sempre nello heap*

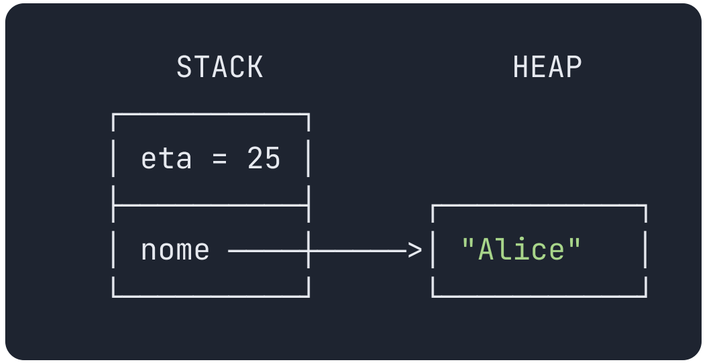
	Stack	Heap
Cosa	Variabili locali, parametri	Oggetti creati con `new`
Vita	Finché il metodo è in esecuzione	Finché qualcuno li riferisce
Gestione	Automatica, a ogni chiamata	Dalla JVM, con il garbage collector
Dimensione	Piccola	Grande

PRIMITIVI VS REFERENCE

Una variabile primitiva contiene il **valore**. Una variabile reference contiene l'**indirizzo** di un oggetto nello heap

```java
// primitivo: il valore è nello stack
int eta = 25;

// reference: sullo stack c'è l'indirizzo,
// l'oggetto è nello heap
String nome = "Alice";
```



L'OPERATORE NEW

`new` fa due cose: crea l'oggetto nello **heap** e restituisce il **riferimento**, che viene salvato nella variabile sullo stack

- *I tipi primitivi (`int`, `double`, `boolean`) non usano `new`*
- *Tutto ciò che si crea con `new` finisce nello heap*
- *Le stringhe letterali sono un caso speciale, lo vedremo tra poco*

```
Studente s = new Studente("Anna", 20);

// 1. new Studente(...) crea l'oggetto
//    nello heap
// 2. il riferimento finisce in s,
//    sullo stack
```

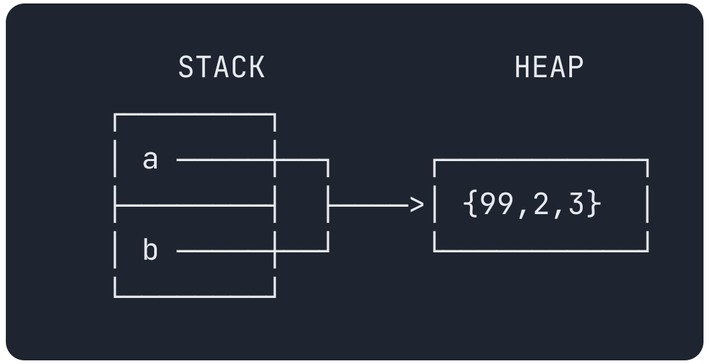
DUE VARIABILI, STESSO OGGETTO

Assegnare un reference a un'altra variabile **non** copia l'oggetto: entrambe puntano allo **stesso** oggetto

- *Modificare tramite* `b` *modifica anche* `a`*: è lo stesso oggetto*
- *Per questo un oggetto passato a un metodo cambia anche fuori*

```
int[] a = {1, 2, 3};
int[] b = a;      // b punta allo stesso array

b[0] = 99;
System.out.println(a[0]);  // 99
```



NULL IN MEMORIA

Una variabile che vale `null` esiste sullo stack, ma non contiene l'indirizzo di nessun oggetto

- *Chiamare un metodo vuol dire seguire il riferimento: se non porta a nessun oggetto, arriva la* `NullPointerException`
- *Solo i reference possono valere* `null`*, i primitivi no*

```
String nome = null;

nome.length();  // NullPointerException
```



IL GARBAGE COLLECTOR

La JVM libera da sola la memoria degli oggetti che nessun riferimento raggiunge più

- *Il **garbage collector** (GC) lavora in background*
- *Non si libera la memoria a mano, come invece in C e C++*
- *Non si sa **quando** il GC passerà: si sa solo che l'oggetto può essere rimosso*

```
Studente s = new Studente("Anna", 20);
s = new Studente("Luca", 22);

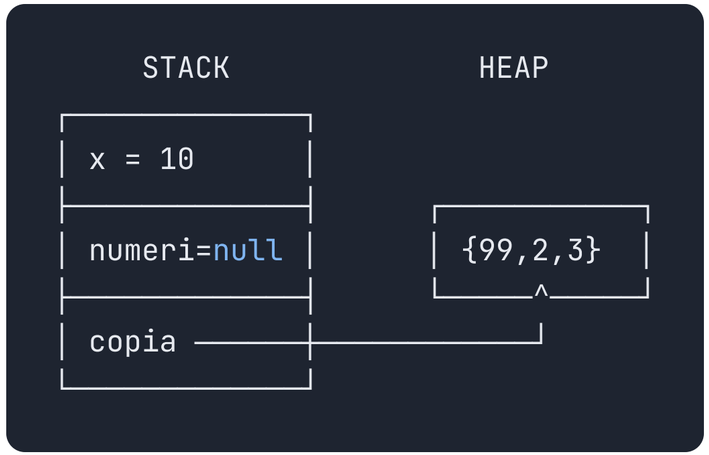
// Anna non ha più riferimenti:
// il GC può rimuoverla
```

ESEMPIO COMPLETO

Seguiamo passo passo cosa succede in memoria

- `numeri` *non punta più all'array, ma* `copia` *sì: l'array resta vivo*

```
int x = 10;
int[] numeri = {1, 2, 3};
int[] copia = numeri;
copia[0] = 99;
numeri = null;
```



QUIZ

Cosa stampa il seguente frammento?

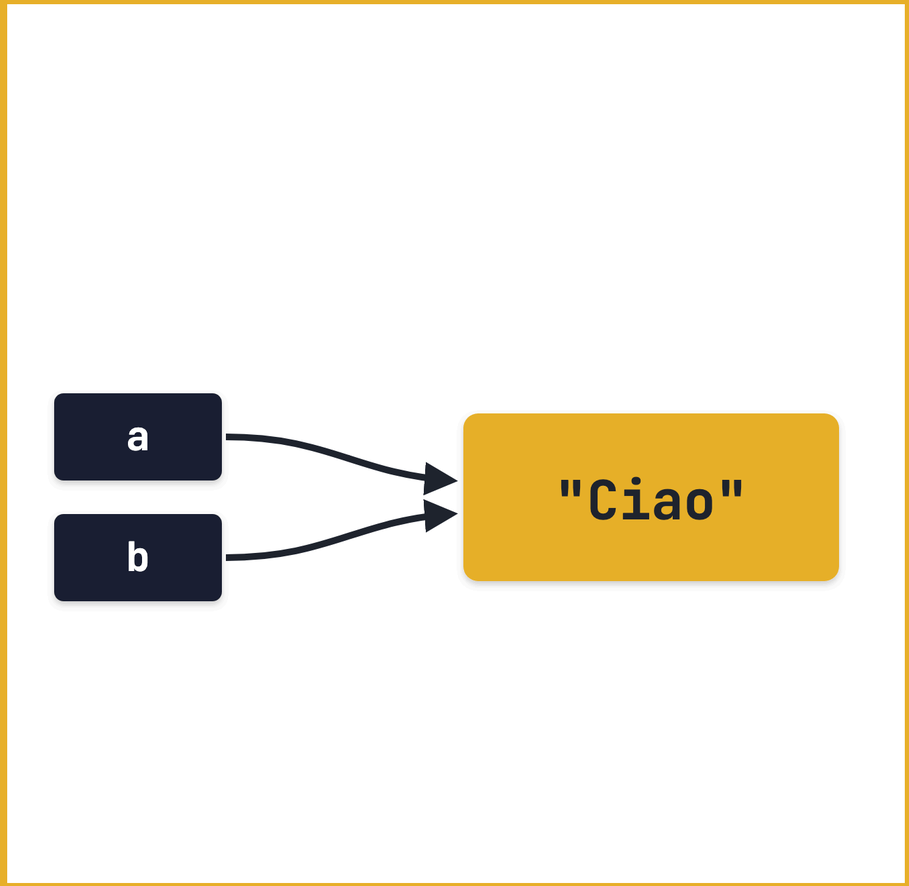
```
int[] a = {10, 20, 30};
int[] b = a;
b[1] = 99;
a = new int[]{5, 6, 7};

System.out.println(b[1]);
System.out.println(a[1]);
```

RISPOSTE

- `b[1]` *stampa* ***99****:* `b` *punta ancora al vecchio array, modificato*
- `a[1]` *stampa* ***6****:* `a` *ora punta a un nuovo array* `{5, 6, 7}`
- *Dopo* `a = new int[]{5, 6, 7}`, `a` *e* `b` *puntano ad array diversi*
- *Il vecchio array* `{10, 99, 30}` *resta vivo perché* `b` *lo riferisce*

02
LE STRINGHE



STRING È UN OGGETTO

`String` non è un tipo primitivo: è una **classe**. Ogni stringa è un oggetto nello heap

- *La* `S` *maiuscola indica una classe, non un primitivo*
- *Ha dei* ***metodi****:* `length()`, `charAt()`, `substring()`
- *Le virgolette sono una scorciatoia per non scrivere* `new`

```
// sembra un primitivo, ma è un oggetto
String saluto = "Ciao";

// forma esplicita, sconsigliata
String altro = new String("Ciao");
```

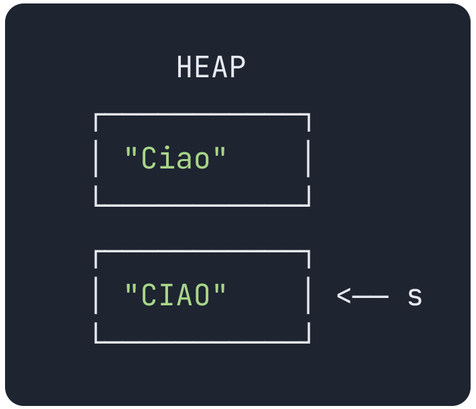
IMMUTABILITÀ

Una volta creata, una stringa **non può essere modificata.** Ogni operazione che sembra modificarla crea un **nuovo oggetto**

- *I metodi di* `String` *restituiscono sempre una* ***nuova*** *stringa*
- *Se non si assegna il risultato, il valore si perde*

```
String s = "Ciao";
s.toUpperCase();
System.out.println(s);   // Ciao

s = s.toUpperCase();
System.out.println(s);   // CIAO
```

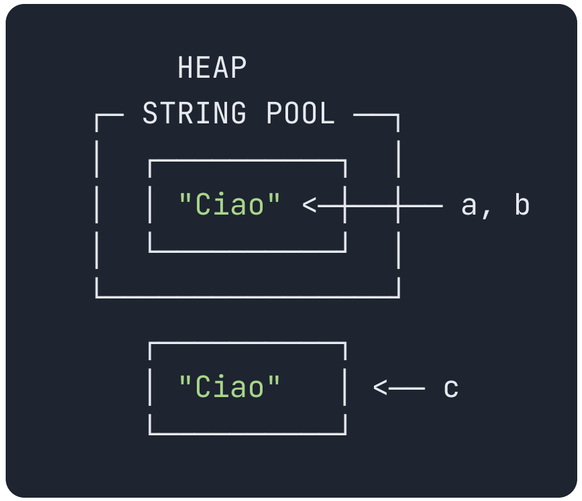


LO STRING POOL

Java mantiene nello heap un'area speciale, lo **String Pool**: i letterali identici condividono lo stesso oggetto

```
String a = "Ciao";
String b = "Ciao";
String c = new String("Ciao");

System.out.println(a == b);  // true
System.out.println(a == c);  // false
```



== VS EQUALS

Con le classi abbiamo usato `equals` per confrontare le stringhe. Con stack, heap e pool si capisce perché

	==	equals()
Confronta	I riferimenti	Il contenuto, carattere per carattere
Vale true	Se è lo stesso oggetto	Se i caratteri sono gli stessi
Con le stringhe	Da evitare	**Sempre**

```
String input = scanner.nextLine();  // l'utente scrive Ciao

if (input.equals("Ciao")) { ... }   // corretto
if (input == "Ciao") { ... }        // false: input non è nel pool
```

METODI PRINCIPALI (1/2)

I metodi che userete più spesso

- *Attenzione:* `length()` *con le parentesi, a differenza di* `array.length`

Metodo	Risultato su `"Ciao Mondo"`
`length()`	`10`
`charAt(0)`	`'C'`
`substring(0, 4)`	`"Ciao"`
`indexOf("Mondo")`	`5`
`contains("Mondo")`	`true`
`toUpperCase()`	`"CIAO MONDO"`
`toLowerCase()`	`"ciao mondo"`

METODI PRINCIPALI (2/2)

- `split()` *restituisce un* ***array di stringhe***
- *Nessun metodo modifica l'originale: restituiscono tutti un nuovo valore*

Metodo	Risultato
`" Ciao ".trim()`	`"Ciao"`
`"Ciao".replace('a', 'o')`	`"Cioo"`
`"a,b,c".split(",")`	`{"a", "b", "c"}`
`"Ciao".startsWith("Ci")`	`true`
`"Ciao".endsWith("ao")`	`true`
`"Ciao".isEmpty()`	`false`
`"".isEmpty()`	`true`

CONCATENAZIONE CON +

L'operatore `+` tra stringhe crea ogni volta una **nuova stringa**. In un ciclo diventa inefficiente

- *Ogni* `+` *crea un nuovo oggetto nello heap*
- *Per concatenazioni ripetute si usa* `StringBuilder`

```
// va bene per poche concatenazioni
String msg = "Ciao" + " " + "Mondo";

// in un ciclo: una stringa nuova
// a ogni giro
String risultato = "";
for (int i = 0; i < 1000; i++) {
    risultato = risultato + i + " ";
}
```

STRINGBUILDER

`StringBuilder` è un oggetto **mutabile**: costruisce la stringa senza creare copie intermedie

Metodo	Cosa fa
`append(x)`	Aggiunge alla fine
`insert(pos, x)`	Inserisce alla posizione
`delete(start, end)`	Rimuove un intervallo
`reverse()`	Inverte il contenuto
`toString()`	Restituisce la `String`

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i).append(" ");
}
String risultato = sb.toString();
```

CONFRONTO CON COMPARETO

`compareTo()` confronta due stringhe in **ordine lessicografico**, cioè l'ordine del dizionario

- *Confronta carattere per carattere, con il codice Unicode*
- *Conta solo il **segno** del risultato, non il valore*
- `compareToIgnoreCase()` *ignora maiuscole e minuscole*

```
String a = "Alice";
String b = "Bob";

int cmp = a.compareTo(b);
// cmp < 0   a viene prima di b
// cmp == 0  sono uguali
// cmp > 0   a viene dopo b

System.out.println(cmp);  // -1
```

QUIZ

Cosa stampa il seguente frammento?

```
String a = "Java";
String b = "Java";
String c = new String("Java");

System.out.println(a == b);
System.out.println(a == c);
System.out.println(a.equals(c));

String d = a.toUpperCase();
System.out.println(a);
```

RISPOSTE

- `a == b` → ***true***: *stesso letterale, stesso oggetto nel pool*
- `a == c` → ***false***: `new` *crea un oggetto separato*
- `a.equals(c)` → ***true***: *stesso contenuto*
- `a` → ***Java***: *la stringa è immutabile, e il nuovo oggetto creato da* `toUpperCase()` *è finito in* `d`

03
FORMATTARE E CONVERTIRE



STRING.FORMAT E PRINTF

`String.format()` crea una stringa formattata, `printf()` la stampa direttamente

- `%.2f` *arrotonda e usa il separatore decimale della lingua del sistema: la virgola su un computer in italiano*

Segnaposto	Tipo	Esempio
`%s`	Stringa	`Alice`
`%d`	Intero	`20`
`%.2f`	Decimale, 2 cifre	`8,57`
`%n`	A capo	

```
String nome = "Alice";
int eta = 20;
double media = 8.567;

String msg = String.format(
    "Studente: %s, età: %d", nome, eta);
System.out.println(msg);

System.out.printf("Media: %.2f%n", media);
// Media: 8,57
```

CONVERSIONI

Convertire tra stringhe e numeri è frequentissimo, per esempio quando si legge l'input dell'utente

- `parseInt` *lancia* `NumberFormatException` *se la stringa non è un numero*
- `String.valueOf()` *funziona con qualsiasi tipo*

```
// da stringa a numero
int n = Integer.parseInt("42");
double d = Double.parseDouble("3.14");

// da numero a stringa
String s1 = String.valueOf(42);
String s2 = String.valueOf(3.14);
String s3 = "" + 42;
```

QUIZ

Cosa stampa il seguente frammento?

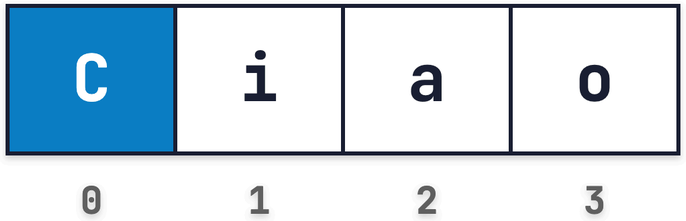
```
String input = "42";
int n = Integer.parseInt(input);
n = n + 8;
String risultato = String.valueOf(n);

System.out.println(risultato + 10);
System.out.println(n + 10);
```

RISPOSTE

- `risultato + 10` → ***5010***: `risultato` *è una stringa, quindi* `+` *concatena*
- `n + 10` → ***60***: `n` *è un* `int`*, quindi* `+` *somma*
- *Regola: se* ***uno*** *dei due operandi è una* `String`*,* `+` *concatena*

04
STRINGHE IN PRATICA



CARATTERE PER CARATTERE

Due modi per scorrere i singoli caratteri di una stringa

- `charAt(i)` *: accesso diretto per indice, come* `array[i]`
- `toCharArray()` *: crea un array di* `char` *, comodo con il for-each*

```
String s = "Ciao";

// con charAt()
for (int i = 0; i < s.length(); i++) {
    System.out.println(s.charAt(i));
}

// con toCharArray()
for (char c : s.toCharArray()) {
    System.out.println(c);
}
```

INVERTIRE UNA STRINGA

Un esercizio classico: costruire la stringa al contrario

- *Il ciclo mostra il procedimento*
- `StringBuilder` *è più breve e non crea stringhe intermedie*

```
String s = "Ciao";

// modo 1: ciclo al contrario
String inversa = "";
for (int i = s.length() - 1; i >= 0; i--) {
    inversa += s.charAt(i);
}
System.out.println(inversa);   // oaiC

// modo 2: StringBuilder
String inversa2 = new StringBuilder(s)
        .reverse().toString();
System.out.println(inversa2);  // oaiC
```

CONTARE LE OCCORRENZE

Contare quante volte un carattere compare in una stringa

- *Lo schema è lo stesso del contatore sugli array*
- *I* `char` *sono primitivi: si confrontano con* `==`

```
String frase = "programmazione";
char cercato = 'a';
int contatore = 0;

for (int i = 0; i < frase.length(); i++) {
    if (frase.charAt(i) == cercato) {
        contatore++;
    }
}
System.out.println(contatore);  // 2
```

CERCARE UNA SOTTOSTRINGA

`indexOf()` restituisce la posizione della prima occorrenza, oppure `-1` se non la trova

- *Prima di usare il risultato, controllare che non sia* `-1`
- `lastIndexOf()` *cerca partendo dalla fine*

```
String frase = "Java è un linguaggio potente";

int pos = frase.indexOf("linguaggio");
System.out.println(pos);    // 10

int pos2 = frase.indexOf("Kotlin");
System.out.println(pos2);   // -1

// cerca dalla posizione 5 in poi
int pos3 = frase.indexOf("a", 5);
System.out.println(pos3);   // 15
```

SPLIT IN PRATICA

`split()` divide una stringa in un array, usando un separatore

- *Utile per leggere dati strutturati, come una riga di un file CSV*
- *I campi numerici si convertono con* `parseInt` *e* `parseDouble`

```
String csv = "Alice,20,8.5";
String[] campi = csv.split(",");

System.out.println(campi[0]);  // Alice
System.out.println(campi[1]);  // 20
System.out.println(campi[2]);  // 8.5

String nome = campi[0];
int eta = Integer.parseInt(campi[1]);
double media = Double.parseDouble(campi[2]);
```

ERRORI COMUNI

I bug più frequenti quando si lavora con le stringhe

Errore	Causa	Correzione
`==` tra stringhe	Confronta i riferimenti	Usare `equals()`
Risultato `-1` usato come indice	`indexOf` non ha trovato nulla	`if (pos != -1)`
Stringa non modificata	`s.toUpperCase()` senza assegnare	`s = s.toUpperCase()`
`NullPointerException`	Metodo su `null`	Controllare `!= null`
`StringIndexOutOfBoundsException`	Indice fuori dalla stringa	Indici da `0` a `length() - 1`
Concatenazione in un ciclo	Crea molti oggetti	Usare `StringBuilder`

QUIZ

Cosa stampa il seguente frammento?

```
String s = "Programmazione";
int count = 0;
for (int i = 0; i < s.length(); i++) {
    char c = s.charAt(i);
    if (c == 'a' || c == 'e' || c == 'i'
     || c == 'o' || c == 'u') {
        count++;
    }
}
System.out.println(count);
System.out.println(s.substring(0, 4));
System.out.println(s.indexOf("gram"));
```

RISPOSTE

- `count` → ***6**: le vocali di "Programmazione" sono o, a, a, i, o, e*
- `s.substring(0, 4)` → ***Prog**: dal carattere 0 al 3, il 4 è escluso*
- `s.indexOf("gram")` → ***3**: "gram" inizia alla posizione 3*

RIEPILOGO

- ***Stack:*** *variabili locali e riferimenti;* ***heap:*** *oggetti*
- ***Reference:*** *più variabili possono puntare allo stesso oggetto*
- ***null e GC:*** `null` *non punta a nulla; il GC libera gli oggetti irraggiungibili*
- ***String:*** *oggetto immutabile; i letterali stanno nel pool*
- ***Confronto:*** `equals()` *per il contenuto,* `==` *per il riferimento*
- ***Metodi:*** `length`, `charAt`, `substring`, `indexOf`, `split`, `replace`, `trim`
- ***StringBuilder:*** *mutabile, per le concatenazioni ripetute*
- ***Conversioni:*** `parseInt`, `parseDouble`, `String.valueOf`, `String.format`

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.0